Software Design Specifications

for

Multi-Agentic RAG Research System

Prepared by:

Team no: 44
Mude Sai Abhinav Chauhan
Vinay Patil
Aishwarya Vodnala
Anish Boppidi
Uppala Siddharth
Supreeth Kunaparaju
Tejas Pothu
Sujato Dutta

Document Information

Title: Multi-Agentic RAG Research System	
Project Manager: Sai Srithaja Nibhabupudi	**Document Version No: 1**
	Document Version Date:09/04/2026
Prepared By: Mude Sai Abhinav Chauhan	**Preparation Date:09/04/2026**

Version History

Version no.	Version Date	Revised by	Description	Filename
1.0	09-04-2026	Mude Sai Abhinav Chauhan	Initial SDS	SDS_Team-44

Table of Contents

1 Introduction

The Software Design Specification describes the architecture and detailed design of the **Multi-Agentic RAG AI Research Scientist system**. The system uses a multi-agent pipeline to process research queries, retrieve academic papers, and generate structured responses using Retrieval-Augmented Generation (RAG).

1.1 Purpose

This document provides a detailed design of the system including architecture, modules, data flow, and interactions between components. It is intended for developers, instructors, and evaluators to understand how the system is implemented.

1.2 Scope

This document covers the design of the multi-agent system including modules such as Planner, Search, Ingestion, Retrieval, Reranking, Reasoning, and Safety agents. It also includes data flow, interfaces, and system behavior.

1.3 Definitions, Acronyms, and Abbreviations

AI – Artificial Intelligence
RAG – Retrieval Augmented Generation
FAISS – Vector database for similarity search
LLM – Large Language Model
API – Application Programming Interface

1.4 References

arXiv API Documentation
FAISS Documentation
Software Engineering Course Material

2 Use Case View

This section describes the main use case of the system, which represents the core functionality of the Multi-Agentic RAG AI Research Scientist.

2.1 Use Case

Actors:

User

Preconditions:

System is running
Internet connection is available

Postconditions:

User receives a structured research response with relevant citations

Basic Flow:

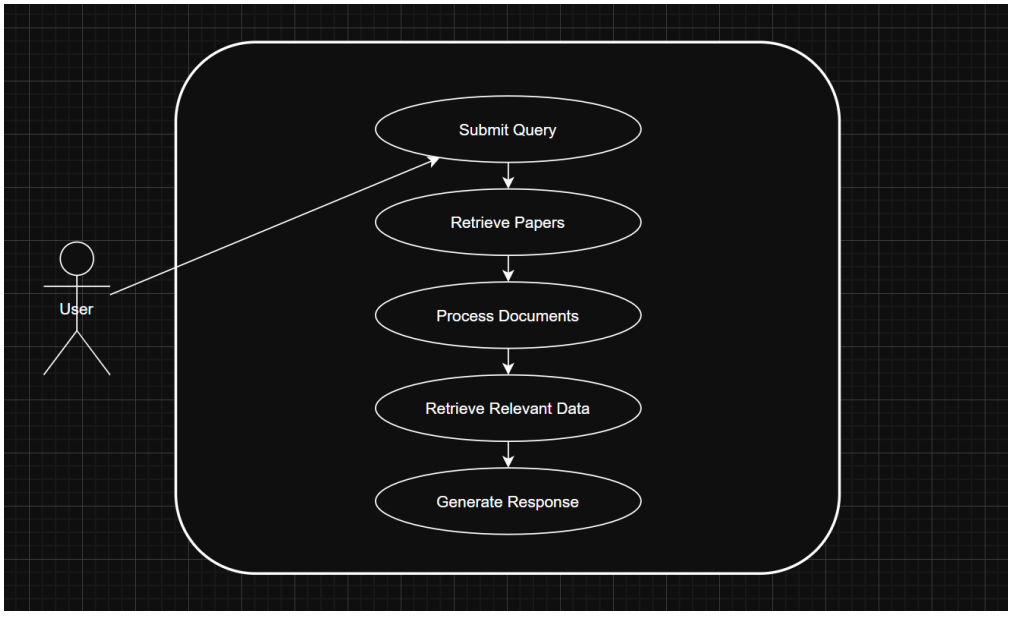
1. User enters a research query
2. Planner Agent analyzes and decomposes the query
3. Search Agent retrieves relevant research papers using arXiv API
4. Ingestion Agent downloads and processes PDFs
5. Retrieval Agent finds relevant document chunks
6. Reranking Agent ranks the retrieved results
7. Reasoning Agent generates a response
8. Safety Agent validates the response and citations
9. System displays final output to the user

Alternative Flow:

If no relevant papers are found, the system asks the user to refine the query

Exceptions:

API failure → System shows error message
PDF processing failure → System skips invalid document



3 Design Overview

This section provides an overview of the architecture and design of the Multi-Agentic RAG AI Research Scientist system. The system follows a **modular multi-agent architecture**, where each component performs a specific task in the research pipeline. The design ensures scalability, maintainability, and clear separation of responsibilities.

3.1 Design Goals and Constraints

Design Goals

- Develop a modular system using multiple specialized agents
- Ensure accurate research output using Retrieval-Augmented Generation (RAG)
- Enable efficient semantic search using FAISS
- Provide structured, citation-based responses
- Maintain scalability for adding new agents or data sources

Constraints

- The system must be implemented using **Python**
- Integration with **arXiv API** for paper retrieval
- Use of **FAISS** for vector similarity search
- Limited by API usage, internet dependency, and compute resources

3.2 Design Assumptions

- Users provide clear and meaningful research queries
- Internet connectivity is available for API access
- External services such as arXiv and LLM APIs are functional
- Research papers are available in readable PDF format

3.3 Significant Design Packages

The system is divided into the following major modules:

- **Planner Agent** – Decomposes user query and defines search strategy
- **Search Agent** – Retrieves research papers using arXiv API
- **Ingestion Agent** – Downloads and processes PDF documents
- **Vector Database (FAISS)** – Stores document embeddings
- **Retrieval Agent** – Performs semantic search on stored data
- **Reranking Agent** – Ranks results using LLM-based scoring
- **Reasoning Agent** – Synthesizes final response from multiple sources
- **Safety Agent** – Validates citations and reduces hallucination

3.4 Dependent External Interfaces

The table below lists the public interfaces this design requires from other modules or applications.

External Application and Interface Name	Module Using the Interface	Functionality/ Description
arXiv API	HTTP API	Used to retrieve research papers
LLM API	API	Used for reasoning and reranking
FAISS Library	Python Library	Used for vector search

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces this design makes available for other applications.

Interface Name	Module Implementing the Interface	Functionality/ Description
Query Input Interface	Planner Agent	Accepts user query
Paper Retrieval Interface	Search Agent	Fetches research papers
Response Output Interface	Reasoning Agent	Returns final structured output

4 Logical View

This section describes the detailed design of the Multi-Agentic RAG AI Research Scientist system. The system follows a **layered multi-agent architecture**, where each module performs a specific function and passes its output to the next module in the pipeline.

4.1 Design Model

The system is composed of multiple interacting modules (agents) in sequential order and each module passes processed data to the next and the design ensures low coupling and high cohesion; each responsible for a specific task:

Main Modules / Classes

- **PlannerAgent**
  - Decomposes user queries
  - Generates search strategy
- **SearchAgent**
  - Retrieves research papers using arXiv API
  - Extracts metadata (title, abstract, PDF link)
- **IngestionAgent**
  - Downloads PDF documents
  - Extracts and preprocesses text
  - Splits documents into chunks

- **VectorStore (FAISS)**
  - Stores embeddings of document chunks
  - Supports similarity search
- **RetrievalAgent**
  - Converts query into embedding
  - Retrieves top-k relevant document chunks
- **RerankAgent**
  - Reorders retrieved chunks using LLM scoring
  - Filters irrelevant results
- **ReasoningAgent**
  - Synthesizes information from multiple sources
  - Generates structured response
- **SafetyAgent**
  - Verifies citations
  - Reduces hallucination

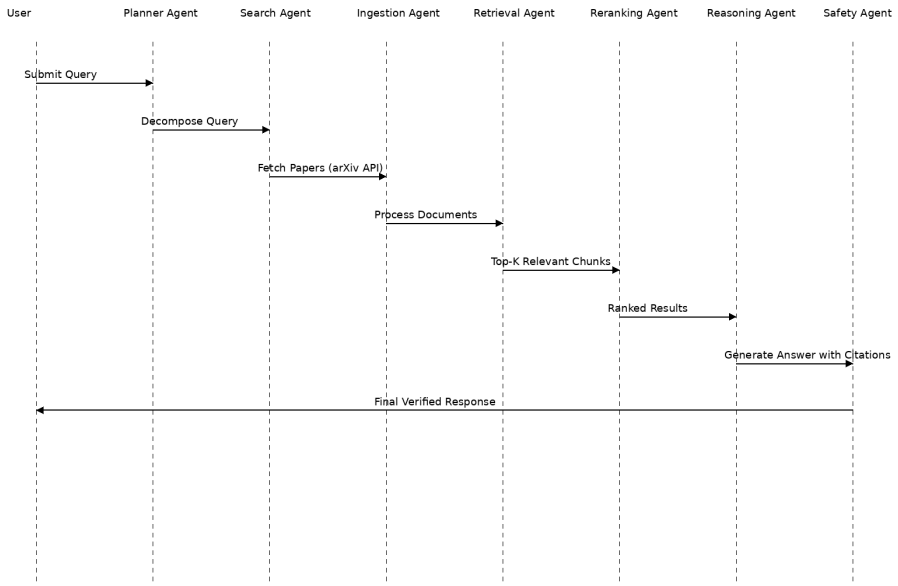
4.2 Use Case Realization

Flow of Execution:

1. User submits a research query
2. **Planner Agent** analyzes and decomposes the query
3. **Search Agent** retrieves relevant papers using arXiv API
4. **Ingestion Agent** downloads and processes PDFs
5. Text is converted into embeddings and stored in **FAISS**
6. **Retrieval Agent** performs semantic search to find relevant chunks
7. **Reranking Agent** ranks results based on relevance
8. **Reasoning Agent** generates a final response
9. **Safety Agent** validates citations and ensures correctness
10. Final structured response is returned to the user

Design Approach

- The system uses a **pipeline-based architecture**
- Each agent performs a **single responsibility**
- Data flows step-by-step ensuring modularity
- The design allows easy addition of new agents



The sequence diagram illustrates the interaction between system components.

5 Data View

This section describes how data is represented, stored, and used in the Multi-Agentic RAG AI Research Scientist system. The system primarily works with **temporary (session-based) data** and uses a vector database for semantic retrieval.

5.1 Domain Model

The system consists of the following main data entities:

- **Query**
  Represents the input provided by the user.
- **Research Paper**
  Contains metadata such as title, authors, abstract, and PDF link retrieved from arXiv.
- **Document Chunk**
  A smaller segment of the research paper created during preprocessing.
- **Embedding Vector**
  Numerical representation of document chunks used for similarity search.
- **Retrieved Results**
  Top-k relevant chunks obtained from the vector database.
- **Response**
  Final structured output generated by the system with citations.

5.2 Data Model (persistent data view)

The system does not rely on traditional databases. Instead, it uses a vector database (FAISS) for storing embeddings of document chunks.

- Data is stored temporarily during each session
- Embeddings are generated dynamically

- No long-term storage is required

5.2.1 Data Dictionary

Data Element	Description
Query	User input text
Paper	Research paper metadata
Chunk	Section of document text
Embedding	Vector representation of text
Retrieved Data	Relevant document chunks
Response	Final generated answer

6 Exception Handling

Common Exceptions and Handling

- **API Failure (arXiv / LLM APIs)**
  If an external API fails or does not respond, the system will retry the request or return an appropriate error message to the user.
- **No Results Found**
  If no relevant research papers are retrieved, the system will prompt the user to refine or modify the query.
- **PDF Download Failure**
  If a research paper cannot be downloaded, the system will skip the document and continue processing remaining papers.
- **PDF Parsing Error**
  If text extraction fails, the system ignores the faulty document and proceeds with available data.
- **Empty Retrieval Results**
  If the Retrieval Agent does not find relevant chunks, the system will notify the user or attempt query reformulation.
- **Invalid or Ambiguous Query**
  If the query is unclear, the system may return a message requesting a more specific input.

Logging and Monitoring

- All errors are logged for debugging and analysis
- System maintains logs for API calls and failures
- Critical errors are reported to the user with clear messages

System Behavior

- Continue execution even if partial failures occur
- Avoid crashing due to individual component errors
- Provide meaningful feedback to the user

7 Configurable Parameters

This table describes the parameters that can be adjusted to control the behavior of the Multi-Agentic RAG AI Research Scientist system.

Configuration Parameter Name	Definition and Usage	Dynamic?
top_k	Number of relevant document chunks retrieved by the Retrieval Agent	Yes
max_results	Number of research papers fetched from arXiv API	Yes
chunk_size	Size of document chunks during preprocessing	Yes
embedding_model	Model used to generate embeddings for text	No
llm_model	Language model used for reasoning and response generation	No
temperature	Controls randomness of LLM responses	Yes
max_tokens	Maximum length of generated response	Yes

8 Quality of Service

This section describes the system's performance, availability, security, and monitoring aspects.

8.1 Availability

The system depends on external services such as the **arXiv API** and LLM APIs. It is designed to remain operational under normal conditions and handle temporary failures gracefully. In case of external service downtime, the system provides appropriate error messages without crashing.

8.2 Security and Authorization

The system ensures secure communication with external APIs using standard protocols (HTTPS). User queries are processed securely, and no sensitive user data is stored. The system prevents unauthorized access and ensures safe handling of generated responses.

8.3 Load and Performance Implications

The system is designed to handle multiple document retrieval and processing operations efficiently. The average response time is expected to be **20–30 seconds**, depending on the number of research papers processed. Performance may vary based on system resources and API response times.

8.4 Monitoring and Control

The system maintains logs for API requests, errors, and system performance. Monitoring helps track response time, retrieval accuracy, and system reliability. These logs can be used for debugging and improving system performance.